

OOPJ - DAY 13

Collection Framework, Interfaces, Lambdas and Functional Programming

Mon, 21 Sept 2026

What this note covers

TreeSet and Map recap, interfaces as contracts, functional interfaces, lambda expressions, anonymous classes, java.util.function interfaces, Comparator lambdas, static imports, Java records, and practice assignments.

Section	Core idea
1. Collection Framework Recap	TreeSet ordering, custom objects, Comparable vs Comparator, Map traversal
2. Interfaces as Contracts	Expose only the operations a caller is allowed to use
3. Functional Interfaces	Exactly one abstract method across the whole interface hierarchy
4. Lambda Expressions	Anonymous function syntax bound to a functional interface
5. Anonymous Classes vs Lambdas	Class-based implementation vs functional implementation
6. Built-in Functional Interfaces	Predicate, Function, Consumer, Supplier, BiPredicate
7. Comparator Lambdas	Concise custom sorting rules
8. Static Imports	Call public static members without the class prefix
9. Records	Compact immutable data carriers
10. Assignments	Practice tasks with hints only

Study cue

Focus on why the compiler can accept a lambda, why TreeSet needs a comparison rule for user-defined objects, and how an interface-typed reference limits visible operations.

1. Collection Framework Recap

TreeSet and natural ordering

TreeSet internally builds a binary tree and uses a sorting rule to place elements. With standard types, that rule is the natural ordering of the type: integers sort in ascending order and strings sort alphabetically.

Rule	TreeSet behavior
Smaller value	Placed to the left of the current node
Greater value	Placed to the right of the current node
Insertion order	Not preserved
Duplicates	Not allowed
Null	Allowed only once according to the supplied notes

Mental model

Think of TreeSet as a sorted set rather than a list. Your insertion order is not the output order; the comparison rule decides where each element belongs.

Why user-defined objects need guidance

If Employee has many data members, TreeSet cannot automatically guess which one should determine ordering. Without a comparison rule, inserting custom objects can result in ClassCastException because the objects do not provide the expected natural ordering.

There are two standard approaches in the notes.

Option 1 - Implement Comparable

```
class Employee implements Comparable<Employee> {
    int id;
    String name;

    @Override
    public int compareTo(Employee other) {
        return this.id - other.id; // ascending by ID
    }
}

TreeSet<Employee> set = new TreeSet<>();
```

Option 2 - Supply a Comparator

```
class EmployeeAgeComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return e1.age - e2.age; // ascending by age
    }
}

TreeSet<Employee> set = new TreeSet<>(new EmployeeAgeComparator());
```

Comparable vs Comparator

Aspect	Comparable	Comparator
Lives where?	Inside the class being ordered	Separate, independent class/object
Method	compareTo(Object o)	compare(Object o1, Object o2)
Arguments	One	Two
Typical use	Default / natural ordering	External or multiple sort criteria

Map recap

A Map stores key-value pairs and uses two generic types: one for the key and one for the value. In the supplied notes, duplicate-key detection is described in terms of equals() and hashCode() on the key, while TreeMap-style ordering of user-defined keys follows the same comparison-rule idea as TreeSet.

```
Map<Integer, Employee> map = new TreeMap<>();
map.put(101, new Employee("Alice"));
map.put(102, new Employee("Bob"));
```

Goal	API
Get a specific value	map.get(key)
Get all values	map.values() -> Collection<V>
Get all keys	map.keySet() -> Set<K>
Get keys and values together	map.entrySet() -> Set<Map.Entry<K,V>>

```
for (Map.Entry<Integer, Employee> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

2. Interfaces as Contracts

An interface defines which methods a caller is allowed to access on an object. The key idea is to pass a reference of the interface type instead of the concrete implementation type. The caller can then invoke only the methods declared by that interface.

```

interface Profile {
    String getUsername();
    String getName();
    void setName(String name);
    String getMobile();
    void setMobile(String mobile);
    void setPassword(String password);
}

class UserProfile implements Profile {
    private String username;
    private String name;
    private String email; // not in Profile - hidden from caller
    private String mobile;
    private String password;

    @Override public String getUsername() { return username; }
    @Override public String getName() { return name; }
    @Override public void setName(String n) { this.name = n; }
    @Override public String getMobile() { return mobile; }
    @Override public void setMobile(String m) { this.mobile = m; }
    @Override public void setPassword(String p) { this.password = p; }
}

```

```

void changeProfile(Profile profile) {
    profile.setName("New Name");
    profile.setMobile("999999999");

    // profile.setEmail(...); // compile error: not in Profile
}

```

Why this matters

The object can have more capabilities than the caller is allowed to use. The reference type controls the visible API, which makes the boundary explicit.

Real-world analogy

A free game version may expose only levels 1-2, while a paid version exposes every level. The underlying system may contain more functionality, but the interface determines what the current user can access.

The same principle with collections

```

Map<Integer, Employee> map = new TreeMap<>();

// map.comparator(); // not accessible through Map reference
// Only Map methods are visible through this reference.

```

This is a practical example of abstraction: the concrete object is a `TreeMap`, but the variable is typed as `Map`, so `TreeMap`-specific methods are hidden through that reference.

3. Functional Interfaces

A functional interface is a regular Java interface with exactly one abstract method. Java 8 introduced this style to support functional programming and lambda expressions.

What else can a functional interface contain?

Method kind	Allowed?	Key point
Abstract	Exactly 1	This is what makes the interface functional
Default	Any number	Has a body
Static	Any number	Belongs to the interface
Private	Any number	Helper methods inside the interface; Java 9+

```

@FunctionalInterface
interface Test {
    void test();

    default void result() {
        System.out.println("default");
    }

    static void initialize() {
        System.out.println("static");
    }

    private void internal() {
        System.out.println("private");
    }
}

```

Remember

@FunctionalInterface is optional. It does not create the functional-interface property; the interface is functional because the total number of abstract methods is exactly one. The annotation acts as a compiler guard.

Child interface rules

Case	Child declaration	Functional?
1	Blank child; inherits one abstract method	Yes
2	Adds a second abstract method	No
3	Adds a default method	Yes
4	Redeclares the same method signature	Yes - still one logical abstract method

Key rule

Count abstract methods across the entire interface hierarchy. The total must be exactly one.

4. Lambda Expressions

A lambda expression is an anonymous function: it has no name, declared return type, or access modifier. A lambda needs a target functional interface so the compiler knows which single abstract method the lambda implements.

PasswordValidator example

```
@FunctionalInterface
interface PasswordValidator {
    boolean validate(String password);
}
```

Four levels of lambda conciseness

```
// Level 1 - explicit type, braces, return
PasswordValidator pv = (String str) -> {
    if (str.length() >= 5) return true;
    else return false;
};

// Level 2 - remove parameter type
PasswordValidator pv = (str) -> {
    if (str.length() >= 5) return true;
    else return false;
};

// Level 3 - remove parentheses for one parameter
PasswordValidator pv = str -> {
    if (str.length() >= 5) return true;
    else return false;
};

// Level 4 - single expression
PasswordValidator pv = str -> str.length() >= 5;
```

Two or more parameters

Round brackets are mandatory. For example: (str1, str2) -> ... is valid, while str1, str2 -> ... is not valid lambda syntax.

Advanced validator: length + @

```
PasswordValidator advancedValidator = str -> {
    if (str.length() >= 5 && str.indexOf('@') != -1) {
        return true;
    }
    return false;
};
```

Expression	Meaning
str.length() >= 5	String contains at least five characters
str.indexOf('@') != -1	The @ character exists somewhere in the string

Expression	Meaning
indexOf(...) == -1	Character was not found

5. Anonymous Classes vs Lambda Expressions

Anonymous class using an abstract class

```

abstract class Display {
    public abstract void show();
}

Display obj1 = new Display() {
    @Override
    public void show() {
        System.out.println("LCD show of Display");
    }
};

Display obj2 = new Display() {
    @Override
    public void show() {
        System.out.println("LED show of Display");
    }
};

```

The compiler internally gives anonymous classes generated names such as Program\$1.class and Program\$2.class. Each anonymous class results in a generated class file.

Anonymous class implementing an interface

```

PasswordValidator pv = new PasswordValidator() {
    @Override
    public boolean validate(String str) {
        return str.length() >= 5;
    }
};

```

Anonymous class	Lambda expression
A class without a source-level name	A function expression without a name
Can implement/inherit from abstract or concrete classes/interfaces	Targets a functional interface only
Can contain any number of methods	Provides the single abstract-method implementation
this refers to the anonymous-class instance	this refers to the enclosing instance
Generates a class file such as Program\$1.class	Does not generate a separate .class file for the lambda in the same model
Can contain instance variables	Does not define instance variables of its own
A separately allocated anonymous class object	Treated as a function object/expression bound to the target type

6. Built-in Functional Interfaces (java.util.function)

Java 8 provides ready-made functional interfaces so you rarely need to define a custom interface for common patterns.

Interface	Method	Takes	Returns
Predicate<T>	test(T t)	1 argument	boolean
Function<T,R>	apply(T t)	1 argument	R
Consumer<T>	accept(T t)	1 argument	void
Supplier<T>	get()	no argument	T
BiPredicate<T,U>	test(T t, U u)	2 arguments	boolean

Predicate<T> - test a condition

```
Predicate<String> validatePassword =
    str -> str.length() >= 5;

System.out.println(validatePassword.test("hello")); // true
System.out.println(validatePassword.test("hi")); // false
```

Why use Predicate<String> instead of raw Predicate?

The generic type provides type safety. Without it, the parameter is treated more generally and may require manual casting, increasing the risk of ClassCastException.

Predicate with a generic class

```
class Data<T> {
    T value1;
    T value2;
}

Data<Integer> objData = new Data<>();
objData.value1 = 10;
objData.value2 = 10;

Predicate<Data<Integer>> checkEqual =
    obj -> obj.value1.equals(obj.value2);
```

Predicate chaining

```
Predicate<String> hasMinLength = str -> str.length() >= 5;
Predicate<String> hasAtSign = str -> str.indexOf('@') != -1;

Predicate<String> combined = hasMinLength.and(hasAtSign);
Predicate<String> negated = hasMinLength.negate();
```

Predicate-based filtering

```
public static void filter(
    List<String> list,
    Predicate<String> condition) {

    for (String data : list) {
        if (condition.test(data)) {
            System.out.println(data);
        }
    }
}

// The lambda can be stored or passed inline.
filter(trainees, str -> str.indexOf('a') != -1);
```

Function<T,R> - accept a parameter, return a value

```
Function<Integer[], Integer> findLargest = arr -> {
    int max = arr[0];
    for (int n : arr) {
        if (n > max) max = n;
    }
    return max;
};

System.out.println(
    findLargest.apply(new Integer[]{3, 7, 1, 9, 4})
); // 9
```

Consumer<T> - accept a parameter, return nothing

```
Consumer<List<String>> displayList = list -> {
    Iterator<String> itr = list.iterator();
    while (itr.hasNext()) {
        System.out.println(itr.next());
    }
};

displayList.accept(trainees);

// Equivalent idea using forEach + method reference:
trainees.forEach(System.out::println);
```

System.out::println is a method reference. println takes one String-like argument here and returns void, which matches Consumer's accept(T) shape.

Supplier<T> - take no parameter, return a value

```
Supplier<Integer> randomDigit =
    () -> (int)(Math.random() * 9);

System.out.println(randomDigit.get());
```

BiPredicate<T,U> - two parameters, return boolean

```
BiPredicate<String, Integer> checkLength =
    (str, len) -> str.length() >= len;

System.out.println(checkLength.test("hello", 3)); // true
```

7. Comparator Lambdas for Sorting

```
List<String> trainees = new ArrayList<>();
trainees.add("Rajal");
trainees.add("Rahul");
trainees.add("Kian");
trainees.add("Aishwarya");
trainees.add("Vivek");
```

Ascending comparator - multi-line form

```
Comparator<String> ascending = (first, second) -> {
    if (first.compareTo(second) > 0) return 1;
    else if (first.compareTo(second) < 0) return -1;
    else return 0;
};

trainees.sort(ascending);
```

Descending comparator

```
Comparator<String> descending = (first, second) -> {
    if (first.compareTo(second) > 0) return -1;
    else if (first.compareTo(second) < 0) return 1;
    else return 0;
};

trainees.sort(descending);
```

Ternary form and inline usage

```
Comparator<String> ascending = (first, second) ->
    first.compareTo(second) > 0 ? 1 :
    first.compareTo(second) < 0 ? -1 : 0;

trainees.sort((first, second) ->
    first.compareTo(second) > 0 ? 1 :
    first.compareTo(second) < 0 ? -1 : 0);
```

Before lambdas

```
class TraineesComparator implements Comparator<String> {
    @Override
    public int compare(String first, String second) {
        return first.compareTo(second) > 0 ? 1 :
            first.compareTo(second) < 0 ? -1 : 0;
    }
}

trainees.sort(new TraineesComparator());
```

Main advantage

Because Comparator is itself a functional interface, lambdas remove the need to create a separate comparator class for every ordering rule such as ID, name, or age.

8. Static Imports

Static imports were introduced in Java 7. They allow public static methods and fields to be referenced without prefixing them with the class name.

```
// Without static import
Date.display();
Date.show();
int x = Date.iStatic;
```

```
// With static import
import static org.pankaj.Date.display;

display();
```

```
// Wildcard static import
import static org.pankaj.Date.*;

display();
show();
iStatic = 1;
```

Ambiguity drawback

```
import static org.pankaj.Date.display;
import static org.pankaj.Data.display;

display(); // compile error: ambiguous
```

Rule	Detail
Same method name	Importing two public static methods with the same name makes a bare call ambiguous
Visibility	Only public static members can be imported

Rule	Detail
Default package	Default-package classes cannot be statically imported because there is no package name to reference

9. Records (Java 14+)

A record is a special Java class for data-carrying objects. It removes common boilerplate and is especially useful for DTO-style classes.

Generated / provided behavior	Record behavior
Constructor	Generated from the declared components
Accessors	Generated as name(), age(), etc.
toString()	Generated
equals()	Generated
hashCode()	Generated
Mutability	Record components are implicitly final

Traditional class vs record

```
// Traditional class: lots of boilerplate for a simple data carrier
class Data {
    private String name;
    private String address;
    private int age;
    private boolean gender;
    // constructor, getters, toString, equals, hashCode...
}

// Record: compact declaration
record Data(
    String name,
    String address,
    int age,
    boolean gender
) {}
```

Records and inheritance

Records can implement interfaces but cannot extend ordinary classes. A record already extends `java.lang.Record`.

```
record Data(
    String name,
    String address,
    int age,
    boolean gender
) implements Comparable<Data> {
    @Override
    public int compareTo(Data other) {
        return this.name.compareTo(other.name);
    }
}
```

Not a normal getter

Record accessors use the component name directly: `objData.name()` and `objData.age()`, not `getName()` or `getAge()`.

```
Data objData =
    new Data("test", "Mumbai", 12, true);

System.out.println(objData.name());
System.out.println(objData.age());
System.out.println(objData);

// Example structured output:
// Data[name=test, address=Mumbai, age=12, gender=true]
```

Overriding toString()

```
record Data(
    String name,
    String address,
    int age,
    boolean gender
){
    @Override
    public String toString() {
        return name;
    }
}
```

10. Assignments and Next Steps

Important

This section intentionally contains questions and hints only. The solution code from the source notes has been removed so the exercises can be completed independently.

Assignment 1 - Supplier OTP Generator

1. Use a `Supplier<String>` lambda that returns a 6-character OTP every time `get()` is called.
2. The OTP format must be: 1 random uppercase vowel + 5 random digits (0-9).

Hints:

- Create a char[] vowel pool containing A, E, I, O, U.
- Choose one vowel with a random index.
- Generate five digits with Math.random().
- Math.random() returns a double in [0.0, 1.0). For digits 0-9, multiply by 10 and cast to int.
- Use StringBuilder or another simple approach to build the final six-character string.

Assignment 2 - Sort a String list alphabetically using a Comparator lambda

3. Create an ArrayList containing at least five names.
4. Write a Comparator<String> lambda for ascending alphabetical order.
5. Pass the comparator to list.sort() or Collections.sort().

Hints:

- Use String.compareTo() inside the lambda.
- Return a positive value, negative value, or zero according to the comparison result.

Assignment 3 - Find the largest number in an integer array using Function

6. Use Function<Integer[], Integer>: the input is an Integer array and the return value is the largest element.
7. Iterate through the array inside the lambda and track the maximum value.
8. Call the function with the array {3, 7, 1, 9, 4}.

Hint: initialize the maximum from the first array element, then scan the remaining elements.

Assignment 4 - Transaction class with Lambda and Collections

Create a Transaction class with these fields:

- int transactionId
- double transactionAmount
- boolean transactionStatus - true = completed, false = pending
- boolean transactionAreas - true = arrears apply, false = no arrears

Skip transactionDate for now, as stated in the source notes.

9. Create five hardcoded Transaction objects and store them in a List or Set.
10. Exercise A: use a Predicate lambda to filter and print all transactions where amount > 5000.
11. Exercise B: use a Predicate lambda to filter and print all transactions where status == false.
12. Exercise C: use a Function lambda to calculate the amount due for each transaction.

Hint for Exercise C:

- When transactionAreas is true, amount due = transactionAmount + 500 + 18% of transactionAmount.
- When transactionAreas is false, amount due = transactionAmount.

Next session

When	Planned topic
Tomorrow at 5 PM	Stream API (uses lambdas and functional interfaces extensively), or another pending topic depending on pace
Pending reminder	Optional - not on syllabus, but needed for web technologies; covered during the recording